Lecture 3: Word2Vec

Andrei Arsene Simion

Columbia University

September 16, 2025

Outline

- Basic Word Embeddings
- CBOW Model
- Skip-Gram Model

Outline

- Basic Word Embeddings
- CBOW Model
- Skip-Gram Model

Word Embeddings Motivation

- **Given**: A corpus $\mathcal{C}$ with sentences
- **Want**: A vector representation for each word a_w such that the vector a_w captures some of the "meaning" of w for each w
- **Ideally**: These vectors have nice mathematical properties
  - If two words w_1, w_2 are related, their vectors are related in the sense that $a_{w_1}^{\mathsf{T}} a_{w_2}$ is high
  - Or, $||a_{w_1} - a_{w_2}||$ is low: we can cluster similar words!
  - Maybe $a_{w_1} - a_{w_2}$ has a "meaning"; $a_{doctor} - a_{man} \sim$ "medical profession person"

Why Word Embeddings: Motivation

- **But Why?** ... There are a few textbook answers:
  - Traditional approaches treated words as independent entities, ignoring their meanings and relationships. Word embeddings, however, represent words as dense vectors in a continuous space, capturing semantic similarities (e.g., "king" and "queen" are close in the embedding space)
  - One-hot encoding creates sparse, high-dimensional vectors (with a dimension for every unique word), which leads to inefficiencies in computation and storage
  - Pretrained embeddings can be used across various NLP applications, reducing the need for large labeled datasets
  - Tasks like sentiment analysis, machine translation, and named entity recognition benefit from embeddings, as they provide richer word representations than simple frequency-based methods
  - Models like Word2Vec utilize context to learn word meanings. This helps in better distinguishing between usage (e.g.,"river bank" vs. "money bank")
    - For Word2Vec, we actually always have 1 embedding for a word, so averaging the two embeddings above is crucial

Word Embeddings: From Str to Int

- Tokenize $\mathcal{C}$ in some way
  - Possible recipe: Take the tokens as the words
  - Take $\mathcal{C}$ and split it on spaces
  - Eliminate rare words?
  - Collapse some words together (running, run, runs)?
  - We now have a vocabulary set $\mathcal{V}$ with a set of words {a, and, the, cat, mat, ...}
  - Each word can be assigned a unique integer, and we have a hash map $stoi = \{w \rightarrow i\}$
  - From this map, we can easily get a reverse hash map $itos = \{i \rightarrow w\}$
- For each $w \in \mathcal{V}$ we can assign a unique integer i so words can be represented as integers
- Naively, we can represent each word as a one-hot 1-0 vector e in $\mathbb{R}^{|\mathcal{V}|}$ so that each element of e is 0 except the ith element which is 1
- Thus, we have $e[stoi[w]] = 1$ and e is zero for all other indices

Word Embeddings Motivation: One-Hot Example

- Suppose $\mathcal{C} =$ "The cat was blue."
- Suppose $\mathcal{V} = \{$"the", "cat", "was", "blue", "."$\}$ so $|\mathcal{V}| = 5$
- Map each word to a unique index so that stoi $= \{$"the" $\rightarrow 0$, "cat" $\rightarrow 1$, "was" $\rightarrow 2$, "blue" $\rightarrow 3$, "." $\rightarrow 4\}$
- Then, a one-hot representation might be
  - $e_{the} = [1, 0, 0, 0, 0]$
  - $e_{cat} = [0, 1, 0, 0, 0]$
  - $e_{was} = [0, 0, 1, 0, 0]$
  - $e_{blue} = [0, 0, 0, 1, 0]$
  - $e_{.} = [0, 0, 0, 0, 1]$

Word Embeddings Motivation: One-Hot Problems

- If $\mathcal{V}$ is a large set, the vectors we get for each word are very large
- The notion of "similarity" is lost, each vector is independent
- Specifically, for any two different words i and j we have

$$a_{w_i}^{\mathsf{T}} a_{w_j} = 0$$

- So, we want better representations and we have some goals
  - **Goal 1**: Try to reduce the size of this space from $\mathbb{R}^{|\mathcal{V}|}$ to something smaller
  - **Goal 2**: Hope that we can find a subspace that encodes the relationships between words so that

$$a_{w_i}^{\mathsf{T}} a_{w_j} = \textit{relationship} \text{ (a number)}$$

SVD ?

- Idea of SVD: express your data as a matrix and "factorize" the matrix so that you capture the "latent" dimensions of some subspace
- Given some (data summary) matrix X, rewrite $X = A\Sigma B^{\intercal}$
  - Can use the co-occourrence matrix X
  - What does it mean for something to co-occur? To in a certain window or distance from each other?
  - Creating X is not easy!

Word Embeddings Motivation: SVD

- Fix a *window size* of 1
- Given the 3 sentences below so $S = \{s_1, s_2, s_3\}$, we can tokenize them into words and get a square matrix of dimension $|\mathcal{V}| \times |\mathcal{V}|$
  - $s_1 =$ "I like driving."
  - $s_2 =$ "I think NLP is cool ."
  - $s_3 =$ "Deep learning NLP is cool ."
- $\mathcal{V} = \{i, like, driving, ., think, nlp, is, cool, deep, learning\}$
- $|\mathcal{V}| = 10$
- Notice that "." is a valid token

Word Embeddings Motivation: SVD

- For any word, again set a unique word integer i via $stoi[w] = i$
- For any two words w_i and w_j let $X[i, j] =$ number of times w_i and w_j are within a window size 1 of each other
- We get the symmetric matrix X:

$$
X = \begin{array}{c} \\ i \\ like \\ driving \\ . \\ think \\ nlp \\ is \\ cool \\ deep \\ learning \end{array}
\begin{array}{c}
\begin{array}{cccccccccc} i & like & driving & . & think & nlp & is & cool & deep & learning \end{array} \\
\begin{pmatrix}
2 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\
1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 0 & 1 & 3 & 0 & 0 & 0 & 2 & 0 & 0 \\
1 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 0 & 0 \\
0 & 0 & 0 & 0 & 1 & 2 & 2 & 0 & 0 & 1 \\
0 & 0 & 0 & 0 & 0 & 2 & 2 & 2 & 0 & 0 \\
0 & 0 & 0 & 2 & 0 & 0 & 2 & 2 & 0 & 0 \\
0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 1 \\
0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 1
\end{pmatrix}
\end{array}
$$

Word Embeddings Motivation: SVD

- Perform SVD on X so that $X = A\Sigma B^{\intercal}$
  - A, Σ, B is $|\mathcal{V}| \times |\mathcal{V}|$
  - Σ is diagonal and has singular values σ_i on it's diagonal
  - Pick some $K < |\mathcal{V}|$ so that $\frac{\sum_{i=1}^{K} \sigma_i}{\sum_{i=1}^{|\mathcal{V}|} \sigma_i}$ is high enough

SVD Word Embeddings

- Approximate X by $\hat{X}$ where we approximate
  - A by $\hat{A}$, where we use only the first K columns
  - Σ by $\hat{\Sigma}$ where we only use the first K rows and K columns
  - B by $\hat{B}$, where we use only the first K rows
- We get an Approximation $X \sim \hat{X} = \hat{A}\hat{\Sigma}\hat{B}^{\intercal}$
- **Note that the $|\mathcal{V}|$ K-dimensional** rows of $\hat{A}$ or/and $\hat{B}$ can be viewed as vector representations for each word
  - Each row of $\hat{A}$ gives us a K-dimensional vector a_w
  - Each row of $\hat{B}$ gives us a K-dimensonal vector b_w
  - We have a vector representation with dimension $K < |\mathcal{V}|$
  - Maybe a_w is what we want? Maybe these vectors have the nice properties we wanted?

SVD Has Problems

- Some problems:
  - X is very sparse
  - Expensive cost to train: $O(|\mathcal{V}|^3)$
  - New words imply that the dimension of the matrix changes very often
  - Very large matrix!
  - X usually is not just co-occurrence counts, we usually need more than this and some hacks to make this work best (so the rows of A capture as much semantic and syntactic information as possible)
- Can use Pearson correlation instead of counts
- Remove function words like "he", "has", "the", "a", etc.
- Use a ramp window: weigh co-occurence matrix by distance instead of just counts
- *Instead of this, let's try something else!*

Word Embeddings Motivation: Towards Word2Vec

- **Idea:** Instead of computing global statistics, we create a model that learns at each iteration and refines what it learns as it progresses
- The main paper (from 2013) below has the core ideas we look at next
  - Efficient Estimation of Word Representations in Vector Space
- After this paper, there were a few generalization and followups but we will probably skip them (ask me if interested!)
  - These Methods: static word embeddings, where each word has 1 representation
  - For example, with $s_1 =$ "On the river bank" and $s_2 =$ "I robbed the bank" there is only 1 vector for "bank"
  - BERT and Modern Embeddings: words don't have 1 embedding (in the above, "bank" gets a different representation depending on its context)

Review: Softmax, Sigmoid, and Cross Entropy

- **Softmax**: given a $\mathbb{R}^{|\mathcal{V}|}$ dimensional vector z, we want to turn z into a probability
- This pre-probability is usually called a logit
- We can map $z \to softmax(z) = (\frac{e^{z_1}}{\sum_{j=1}^{|\mathcal{V}|} e^{z_j}}, \ldots, \frac{e^{z_{|\mathcal{V}|}}}{\sum_{j=1}^{|\mathcal{V}|} e^{z_j}})$
- This preserves the order among the components, and is differentiable!
- $z \to max(z)$ is NOT differentiable!
- **Sigmoid**: given a real number, we can map it to a probability via $z \to \sigma(z) = \frac{1}{1+e^{-z}}$
- This is used in Logistic Regression, and we will need it here as well

Review: Softmax, Sigmoid, and Cross Entropy

- **Cross-Entropy**: Give two distributions q and p over $\mathcal{V}$, how do we measure how close they are?
- The Cross-Entropy can be used measure the dissimilarity

$$Cross - Entropy(p, q) = -\sum_{i=1}^{|\mathcal{V}|} q_i \log p_i$$

- If q is fixed, what is the optimal p? You'll get $p = q$
- To see this, minimize

$$\mathcal{L}(p) = -\sum_{i=1}^{|\mathcal{V}|} q_i \log p_i$$

  subject to $\sum_{i=1}^{|\mathcal{V}|} p_i = 1$
- Typically for us q is the y, a one-hot vector and p is the Softmax of some Logit that a model predicts (a vector)

Review: Cross Entropy and Softmax vs using Max

- Generally, if our model is trying to predict 1 of $|\mathcal{V}|$ classes and we return a vector z of dimension $|\mathcal{V}|$, we can entice the model to give the highest component value to z to the true class y_j by considering the loss

$$\mathcal{L} = Cross - Entropy(p, y) = -\sum_{i=1}^{|\mathcal{V}|} y_i \log p_i = -\log p_j$$

  where $p = softmax(z)$
- Consider an alternative where we'd try

$$\mathcal{L} = -1_{\{argmax\{z\}==j\}}$$

  so we directly involve the max and entice z to put the highest value of z_j so that we "predict" the right class
- The last loss is very direct, but much more numerically difficult to optimize ... What's the gradient? When is it nonzero?

Outline

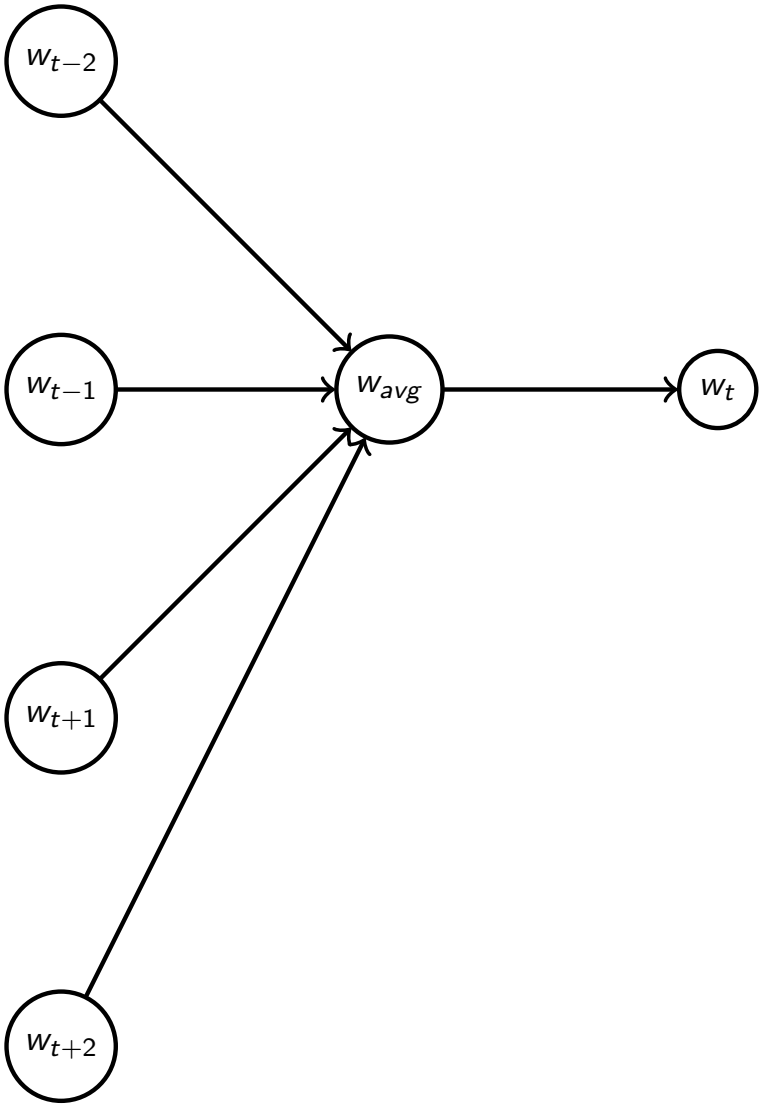
- Basic Word Embeddings
- CBOW Model
- Skip-Gram Model

Word2Vec At A High Level

- A high level summary:
  - 2 models: CBOW and Skip-Gram
    - CBOW: predict a center word from the words around it
    - Skip-Gram: predict words around a center word
  - 2 tricks: Negative Sampling and Hierarchical Softmax
    - Both these methods try to solve for the issue that $|\mathcal{V}|$ is a very large and leads to slow optimization as we'll see
- Both models are built on the idea: "You shall know a word by the company it keeps" (Distributional Hypothesis - Harris 1954)
- I.e. Words that are close together or used in the same way tend to mean the same thing

Word Embeddings: CBOW

- The goal of CBOW is to predict the missing center word from the neighboring context words



Word Embeddings: CBOW

- We need to model $p(w_{output}|w_{context})$
- Given a sequence "student studied NLP into the" we can let NLP be the output word and "student, studied, into, the" be the context
- Define two matrices $B \in \mathbb{R}^{|\mathcal{V}| \times d}$ and $A \in \mathbb{R}^{|\mathcal{V}| \times d}$ where d represents the embedding space
- If we have the one-hot representation of a word, $e_w \in \mathbb{R}^{|\mathcal{V}|}$, then

$$B^{\intercal} e_w = b_w \in \mathbb{R}^d$$

  can be viewed as dimensional vector representation of w
  - We also have

$$A^{\intercal} e_w = a_w \in \mathbb{R}^d$$

Word Embeddings: CBOW Algorithm

- Given a set of documents $\mathcal{C}$, break up $\mathcal{C}$ into sentences and tokenize $\mathcal{C}$ to get the vocabulary $\mathcal{V}$
- Fix a window size m so for each word we look m words to the right and left
- Example: if $s = $ "This is a short sentence." and $m = 1$ we get
  - ["*this*", "*is*", "*a*"]
  - ["*is*", "*a*", "*short*"]
  - ["*a*", "*short*", "*sentence*"]
  - ["*short*", "*sentence*", "."]

Word Embeddings: CBOW Algorithm

- For each center out word w_o and context words, gather (x, y) pairs like

$$((w_{o-m}, \ldots, w_{o-1}, w_{o+1}, \ldots, w_{o+m}), w_o)$$

  - $["a", "short", "sentence"]$ becomes $[("a", "sentence"), "short"]$ and $w_o = "short"$
- For each context word, get the one-hot representations $e_w \in \mathbb{R}^{|\mathcal{V}|}$
- For each context word, get the vector representations $(A^\intercal e_{w_{o-m}}, A^\intercal e_{w_{o-1}}, A^\intercal e_{w_{o+1}}, \ldots, A^\intercal e_{w_{o+m}})$ so that

$$A^\intercal e_w = a_w \in \mathbb{R}^d$$

etc.

Word Embeddings: CBOW Algorithm

- Average the vectors $a_{avg} = \frac{a_{w_{o-m}} + \ldots + a_{w_{o+m}}}{2m} \in \mathbb{R}^d$
- Generate logit scores $z = Ba_{avg} \in \mathbb{R}^{|\mathcal{V}|}$
- Turn the scores into probabilities $p = \text{softmax}(z) \in \mathbb{R}^{|\mathcal{V}|}$
- We hope that these probabilities p match the true probabilities y and we have the one-hot representation of the output word $e_{w_o} = y$
- I.e. We hope that $p_{w_o} \sim 1$ and p is zero elsewhere

Word Embeddings: CBOW Algorithm

- Given that we want $p \sim y$, we can focus on minimizing the loss (Cross-Entropy)

$$\mathcal{L}(A, B) = -\sum_{i=1}^{|\mathcal{V}|} e_i \log p_i$$

- Note that, since e is the one-hot vector of w_o; the above is actually just one term of the type $-\log p_j$ where j is the index of w_o in $\mathcal{V}$; $stoi[w_o] = j$
- If $p_j \sim 1$, as it should be, we get a 0 penalty
- If $p_j \sim 0$, we get a $+\infty$ penalty!
- If $p_j \sim \frac{1}{10}$, we get a loss of $-\log \frac{1}{10} = 4.605$
  - *In either of the last two cases, we've given too low a probability to something that should have a high probability so the loss will be large*

Word Embeddings: CBOW Algorithm

- The loss of a general output word w_o and fixed window around it is:

$$
\begin{aligned}
\mathcal{L}(A, B) &= -\log p(w_o | w_{o-m}, \ldots, w_{o-1}, w_{o+1}, \ldots, w_{o+m}) \\
&= -\log p(w_o | a_{avg}) \\
&= -\log \frac{\exp b_{w_o}^{\mathsf{T}} a_{avg}}{\sum_{w \in \mathcal{V}} \exp b_w^{\mathsf{T}} a_{avg}} \\
&= -b_{w_o}^{\mathsf{T}} a_{avg} + \log \sum_{w \in \mathcal{V}} \exp b_w^{\mathsf{T}} a_{avg}
\end{aligned}
$$

- Generally, the total loss of this model is the loss across the entire training data by windowing in this way and averaging over the number of windows
- The parameters of this model are A and B
- We can use SGD to update the parameters of this model
  - $A_{new} \leftarrow A_{old} - \eta \nabla_A \mathcal{L}(A_{old}, B_{old})$
  - $B_{new} \leftarrow B_{old} - \eta \nabla_B \mathcal{L}(A_{old}, B_{old})$

Word Embeddings: CBOW Gradients

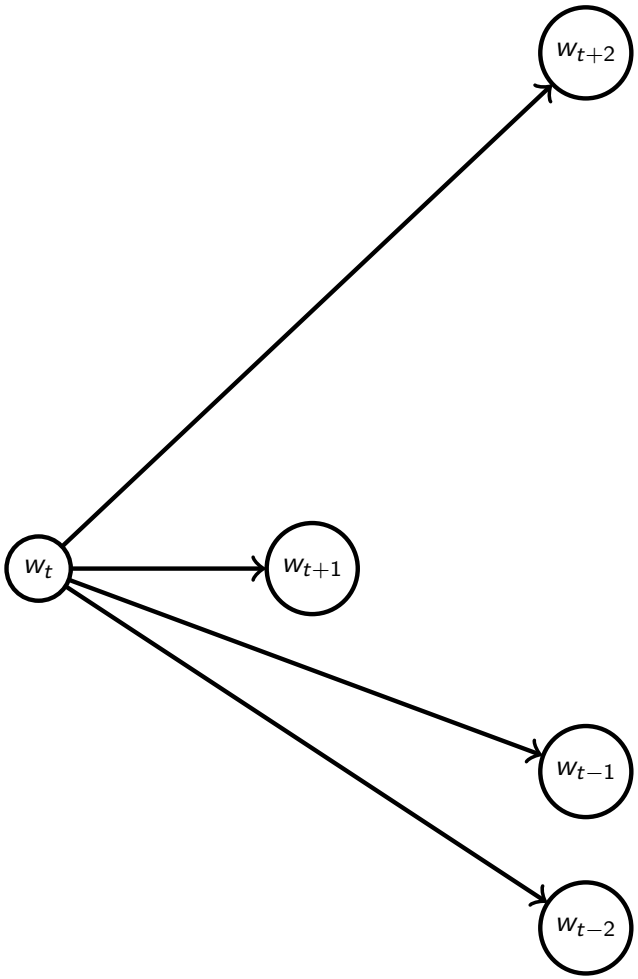
- The loss of a general model looks like an average of $Cross - Entropy$, where we divide by the number of windows (we'll see how to get the training data when we look at code)
- I'll ask you to write this down in the next HW

Outline

- Basic Word Embeddings
- CBOW Model
- Skip-Gram Model

Word Embeddings: Skip-Gram

- Another approach takes the reverse of the CBOW model: given a center word, predict the words around this word
- As with CBOW, you have parameter embedding matrix A, B as before



Word Embeddings: Skip-Gram Model

- Again: given a set of sentences $\mathcal{C}$, tokenize at the word level and get the vocabulary $\mathcal{V}$ and fix a window size m
- Get, from all sentences, all center (context) input words c and output words o pairs, gathering pairs like

$$(w_c, w_{c-m}), \ldots, (w_c, w_{c+m})$$

  so that output words are within m distance of the context word w_c (on either side)
  - Note that in this case the center word is a feature (x) and the output word is the target (y)
- For each context center word, get the one-hot representations $e_{w_c} \in \mathbb{R}^{|\mathcal{V}|}$
- For each context center word, get the vector representations

$$a_{w_c} = A^\intercal e_{w_c} \in \mathbb{R}^d$$

Word Embeddings: Skip-Gram Model

- Generate a score $z = Ba_{w_c} \in \mathbb{R}^{|\mathcal{V}|}$
- Turn the scores into probabilities $p = \text{softmax}(z) \in \mathbb{R}^{|\mathcal{V}|}$
- We have that $p_{w_{c-m}}, p_{w_{c-m+1}}, \ldots, p_{w_{c+m-1}}, p_{w_{c+m}}$, are the probabilities of observing the context words given the center word w_c
- As with CBOW, we want the probabilities above to match the one-hot vectors $e_{w_{c-m}}, e_{w_{c-m+1}}, \ldots, e_{w_{c+m-1}}, e_{w_{c+m}}$
- Specifically, for a fixed center word w_c we have loss terms equal to

$$\mathcal{L}(A, B) = -\sum_{i=-m, i\neq 0}^{m} e_{w_{c+i}} \log p_{w_{c+i}}$$

Word Embeddings: Skip-Gram Model

- We can spell-out the above more by noticing that in our model we want to minimize:

$$
\begin{aligned}
\mathcal{L}(A, B) &= -\log p(w_{c-m}, w_{c-m+1}, \ldots, w_{c+m-1}, w_{c+m} | w_c) \\
&= -\log \prod_{i=-m, i\neq 0}^{m} p(w_{c+i} | w_c) \\
&= -\log \prod_{i=-m, i\neq 0}^{m} \frac{\exp b_{w_{c+i}}^{\mathsf{T}} a_{w_c}}{\sum_{j=1}^{|\mathcal{V}|} \exp b_{w_j}^{\mathsf{T}} a_{w_c}} \\
&= -\sum_{i=-m, i\neq 0}^{m} b_{w_{c+i}}^{\mathsf{T}} a_{w_c} + 2m \log \sum_{j=1}^{|\mathcal{V}|} \exp b_{w_j}^{\mathsf{T}} a_{w_c}
\end{aligned}
$$

Word Embeddings: Skip-Gram Model

- Note that in the above, we've used the *Naive Bayes'* assumption $p(A, B|C) = p(A|C)p(B|C)$ repeatedly
- Note that in actuality we have that

$$\mathcal{L}(A, B) = -\sum_{i=-m, i\neq 0}^{m} \log p(b_{w_{c+i}}|a_{w_c})$$

is the sum of $2m$ terms

$$\sum_{i=-m, i\neq 0}^{m} Cross - Entropy(e_{c+i}, softmax(Ba_{w_c}))$$

Skip-Gram Gradients

- So, a general term of the Skip-Gram objective looks like

$$\mathcal{L}(A, B) = -\log p(b_{w_o}|a_{w_c})) = -b_{w_o}^{\mathsf{T}} a_{w_c} + \log \sum_{w \in \mathcal{V}} \exp b_w^{\mathsf{T}} a_{w_c}$$

- We have (HW!)
  - $\frac{\partial \mathcal{L}}{\partial b_{w_o}} = -a_{w_c} + \frac{a_{w_c} \exp b_{w_o}^{\mathsf{T}} a_{w_c}}{\sum_{u \in \mathcal{V}} \exp b_u^{\mathsf{T}} a_{w_c}}$
  - $\frac{\partial \mathcal{L}}{\partial a_{w_c}} = -b_{w_o} + \sum_{w \in \mathcal{V}} b_w \frac{\exp b_w^{\mathsf{T}} a_{w_c}}{\sum_{u \in \mathcal{V}} \exp b_u^{\mathsf{T}} a_{w_c}}$
    - Notice that this is $E[b_w] - b_{w_o}$ where the expectation is using $p(w) = \frac{\exp b_w^{\mathsf{T}} a_{w_c}}{\sum_{u \in \mathcal{V}} \exp b_u^{\mathsf{T}} a_{w_c}}$
    - Gradient = Expected Guess - Hard Guess

Skip-Gram and CBOW issues

- One big problem with the Skip-Gram and the CBOW model is the (normalizing) summation $\sum_{w \in \mathcal{V}} \exp b_w^\mathsf{T} a_{w_c}$
- This term is the Softmax denominator so if we want to use the Softmax as is we need it
- Second, because we use the Softmax, this term shows up in the gradient computations
- Overall, computing a gradient is then a $O(|\mathcal{V}|)$ computation, which is expensive
- **Idea:** Can we do better via some approximation method?

Softmax Meaning

- Let $z \in \mathbb{R}^{|\mathcal{V}|}$ be some vector and $p = softmax(z)$
- Fix some j and consider p_j when we have $y_j = 1$ and $y_i = 0$ if $i \neq j$
- What does p_j mean?
  - It means j is the true class, a positive sample
  - Also, from the denominator, we know that there are $|\mathcal{V}| - 1$ other classes, and they are negative with respect to p_j
- Using $p_j = \frac{e^{z_j}}{e^{z_1} + \ldots + e^{z_{|\mathcal{V}|}}}$, a Softmax-based objective basically tries to make the logit z_j as big as possible relative to the other logits $z_1, z_2, z_{j-1} \ldots z_{j+1}, z_{|\mathcal{V}|}$
- For us, the logit we want to push up is $b_{w_o}^{\mathsf{T}} a_{w_c}$ since w_c predicts w_o
- Also, want to push down $b_{w_k}^{\mathsf{T}} a_{w_c}$ for any $k \neq o$
- $z = Ba_{w_c} \in \mathbb{R}^{|\mathcal{V}|}$ here ...

Negative Sampling

- So, consider the Skip-Gram model with center word w_c and some context word w_o; we know that $p(w_o|w_c)$ should be nonzero and we want it to be high
- Thus, in this setting, we have that (w_c, w_o) is a positive $(+1)$ sample, so we can use the sigmoid to approximate

$$p(w_o|w_c) \sim \sigma(b_{w_o}^{\mathsf{T}} a_{w_c}) = \frac{1}{1 + \exp -b_{w_o}^{\mathsf{T}} a_{w_c}}$$

- Similarly, we sample K words w_k that are not in the context and we know these have a negative label (-1), so

$$p(w_k|w_c) \sim 1 - \sigma(b_{w_k}^{\mathsf{T}} a_{w_c}) = \sigma(-b_{w_k}^{\mathsf{T}} a_{w_c}) = \frac{1}{1 + \exp b_{w_k}^{\mathsf{T}} a_{w_c}}$$

Negative Sampling

- In particular, we replace

$$p(w_o|w_c) = \frac{\exp b_{w_o}^{\mathsf{T}} a_{w_c}}{\sum_{j=1}^{|\mathcal{V}|} \exp b_{w_j}^{\mathsf{T}} a_{w_c}}$$

by

$$p(w_o|w_c) = \left(\frac{1}{1+\exp -b_{w_o}^{\mathsf{T}} a_{w_c}}\right) E_{w_k \sim p_{sample}(w)}\left[\prod_{k=1}^{K} \frac{1}{1+\exp b_{w_k}^{\mathsf{T}} a_{w_c}}\right]$$

- We sample each word w_k from the empirical distribution of the words in the corpus
- With high probability, for a given w_c, these words will not be in the context of w_c

Word Embeddings: Negative Sampling

- With this approximation, we replace the original term in the objective

$$-\log p(w_o|w_c) = -b_{w_o}^{\mathsf{T}} a_{w_c} + \log \sum_{j=1}^{|\mathcal{V}|} \exp b_{w_j}^{\mathsf{T}} a_{w_c}$$

by

$$-\log \frac{1}{1+\exp -b_{w_o}^{\mathsf{T}} a_{w_c}} - E_{w_k \sim p_{sample}(w)}[\sum_{k=1}^{K} \log \frac{1}{1+\exp b_{w_k}^{\mathsf{T}} a_{w_c}}]$$

- **Instead of having $O(|\mathcal{V}|)$ complexity, we now have $O(K)$**
- Note that this is easily written with sigmoids (σ) as

$$-\log \sigma(b_{w_o}^{\mathsf{T}} a_{w_c}) - E_{w_k \sim p_{sample}(w)}[\sum_{k=1}^{K} \log \sigma(-b_{w_k}^{\mathsf{T}} a_{w_c})]$$

Word Embeddings: Negative Sampling

- We replaced the original term in the objective

$$-\log p(w_o|w_c) = -b_{w_o}^{\mathsf{T}} a_{w_c} + \log \sum_{j=1}^{|\mathcal{V}|} \exp b_{w_j}^{\mathsf{T}} a_{w_c}$$

by

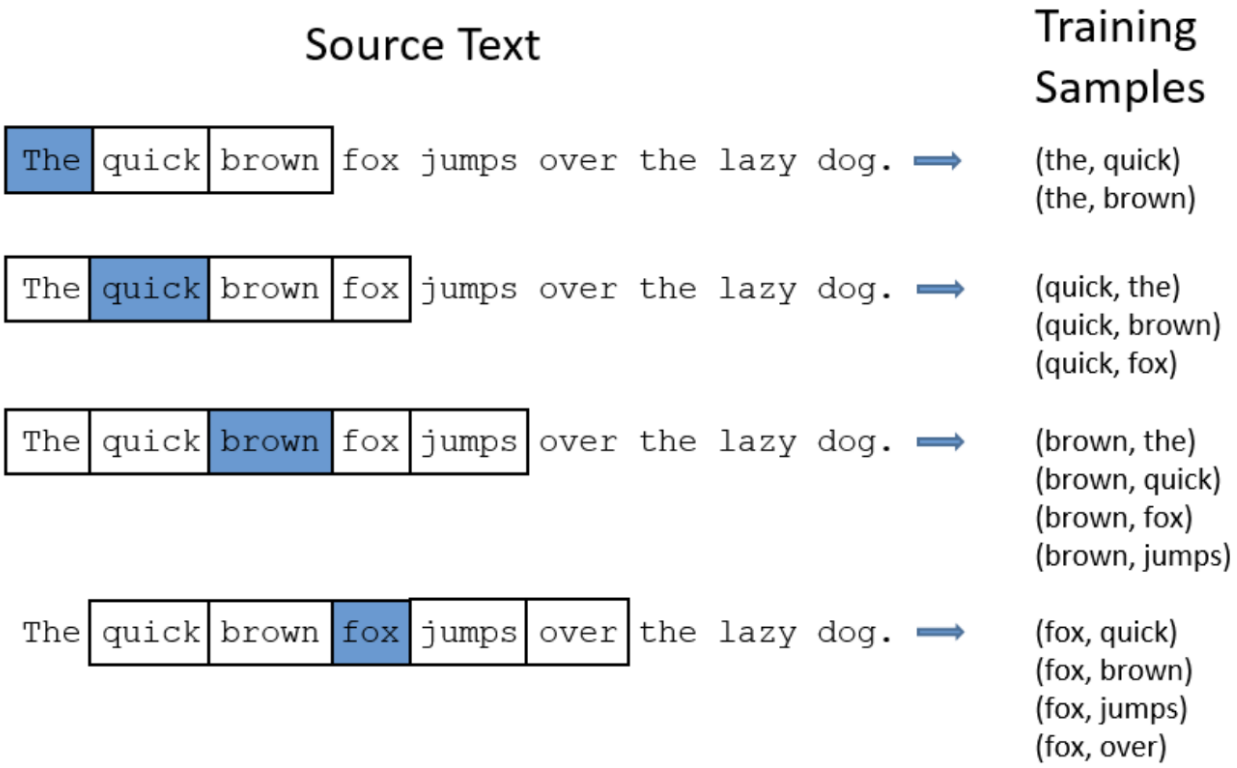
$$-b_{w_o}^{\mathsf{T}} a_{w_c} + \log\left(1 + \exp b_{w_o}^{\mathsf{T}} a_{w_c}\right) + \sum_{k=1, w_k \sim p_{sample}(w)}^{K} \log\left(1 + \exp b_{w_k}^{\mathsf{T}} a_{w_c}\right)$$

Word Embeddings: Negative Sampling + Dropout

- Sampling the K negative samples is an art
- The authors of Skip-Gram used $p_{sample}(w) \propto freq(w)^{\frac{3}{4}}$
- The 3/4 has the effect of pushing less frequent words up more than frequent words
  - "cactus": $0.09 \rightarrow 0.09^{3/4} = 0.16$
  - "the": $0.9 \rightarrow 0.9^{3/4} = 0.92$
- Before doing anything through, the authors also drop words from the corpus
- $p_{keep}(w) = (\sqrt{\frac{z(w)}{0.001} + 1})\frac{0.001}{z(w)}$ where $z(w) \propto freq(w)$ in the original corpus
- After you drop words, you compute p_{sample} as above

Why drop words?

- Words like "the" don't offer much to help us understand the word "fox"
- We have enough samples with "the" as the center words to help us understand "the"



Why does this work?

- Let's focus on CBOW and use a window of size 3
- In this case, we have (u, v, w) as 3 words and we use the average A representations of u and w to modify v's B representation so the inner product we want is large

$$\frac{(a_u + a_w)^\top b_v}{2}$$

- So I have a sentence like "the queen ruled" and another sentence like "the king reigned"
- Given this, we have that the A vectors of $(\textit{the}, \textit{ruled})$ (averaged) is enticed to be like the B vector of *queen*
- We also have that the A vectors of $(\textit{the}, \textit{reigned})$ (averaged) is enticed to be like the B vector of *king*
- We do this many times, and get that the B vectors of "queen" and "king" are both enticed to be similar, since they are similar to the same words
- Doing this over and over, we have that for any word it's $(a + b)/2$ vector is similar to all semantically similar words

Hierarchical Softmax : Another trick to bypass $O(|\mathcal{V}|)$

- Hierarchical Softmax is an efficient way of computing Softmax. The model uses a binary tree to represent all words in the vocabulary. The $\mathcal{V}$ words must be leaf units of the tree
- For each leaf unit, there exists a unique path from the root to the unit; and this path is used to estimate the probability of the word represented by the leaf unit
- Main advantage: we replace the $O(|\mathcal{V}|)$ Softmax cost with $O(\log |\mathcal{V}|)$
- The speed of this method depends on how the binary tree is built and words are associated with the leaf nodes; the paper uses a binary Huffman tree
- Good project? This layer is not available in PyTorch by default

Geometry of Vectors

- One neat thing about these vectors is they seem to have vector properties
- $Italy - Rome + Lisbon = Portugal$!
- But they also have issues: $Doctor - Man + Women = Nurse$ (Bias!)

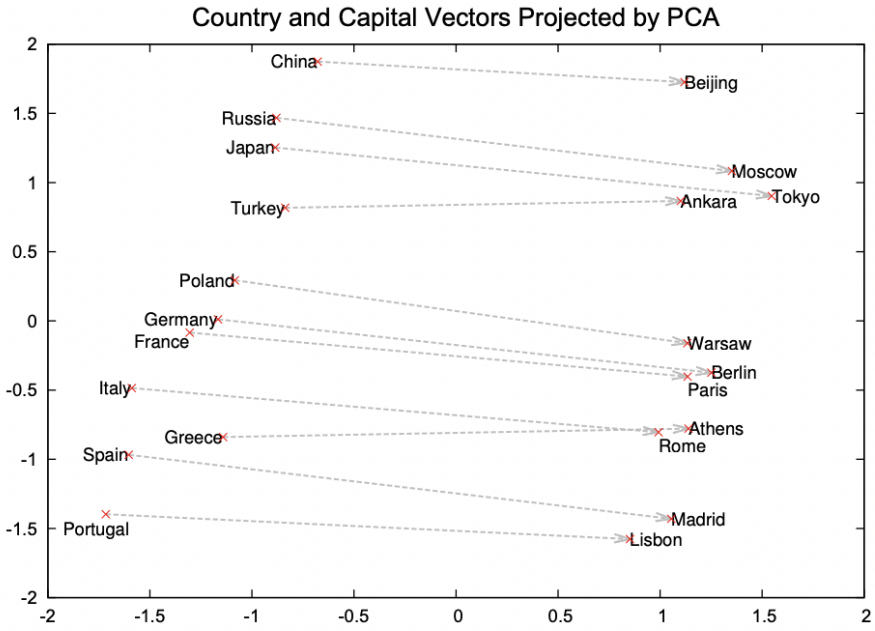


Figure 2: Two-dimensional PCA projection of the 1000-dimensional Skip-gram vectors of countries and their capital cities. The figure illustrates ability of the model to automatically organize concepts and learn implicitly the relationships between them, as during the training we did not provide any supervised information about what a capital city means.

Geometry of Vectors

- Lot's of research went into trying to remove bias
- One great paper is in the references

References

- Efficient Estimation of Word Representations in Vector Space
- Distributed Representations of Words and Phrases and their Compositionally
- Chris McCormick's BLOG on Word2Vec
- CBOW Negative Sampling
- Bias in Word Vectors
- Great paper on Word Vector Bias